

DIGITAL TWIN

A Report for

DIGITAL TWIN PROCESS FOR A PUMP SYSTEM

Presented By

LEMUEL HORNSBY-ODOI

Submitted To

SARA FERRENO GONZALEZ

&

ALICIA MUNIN DOCE

Contents

CHAPTER 1: INTRODUCTION	3
1.1 Background of report.....	3
1.2 Objectives of the report.....	4
1.3 Motivation	4
1.4 Scope	4
CHAPTER 2: SYSTEM DESCRIPTION	5
CHAPTER 3: DIGITAL TWIN ARCHITECTURE.....	7
3.1 Overview	7
3.2 Data Acquisition layer	7
3.3 Simulation layer.....	8
3.4 Cloud and Visualization Layer	8
CHAPTER 4: FMU MODEL DEVELOPMENT AND SIMULATION	9
4.1 Overview	9
4.2 System Modeling in Amesim	9
4.3 Experimental Data for Model Validation.....	10
4.4 FMU Export and Simulation Workflow	10
4.5 Pump Curve Development.....	11
CHAPTER 5: REAL TIME INTEGRATION	12
5.1 Integration Workflow.....	12
5.2 Code Structure.....	12
5.3 Simulation Modes	12
5.4 Data Buffering and Synchronization	12
CHAPTER 6: RESULTS AND LIMITATIONS.....	13
6.1 Results	13
6.2 Limitations of the Model.....	14
CHAPTER 7: CONCLUSIONS AND FUTURE WORK	16
7.1 Conclusions	16
7.2 Future Work	16

CHAPTER 1: INTRODUCTION

Digital Twin technology enables the creation of a virtual replica of a physical system that is continuously updated with real-time data. This project develops a Digital Twin of a pump system, integrating a laboratory pump rig with a physics-based model built in Siemens Amesim and exported as a Functional Mock-up Unit (FMU). The twin synchronizes real-time experimental data—such as flow rate, inlet and outlet pressures, and pump speed—with simulated results to enable direct performance comparison and system monitoring.

The physical setup consists of a centrifugal pump connected to sensors interfaced through a National Instruments (NI-DAQ) system. The measured voltages are calibrated to engineering units using predefined mathematical relationships. On the virtual side, the Amesim model represents the hydraulic behavior of the pump system. The FMU, executed in Python through the FMPy library, receives live inputs from the sensors and returns simulated outputs of flow rate and pressure difference.

The key objective of this work is to establish a real-time connection between the physical and virtual domains, achieving synchronized visualization of measured and simulated data. The Python-based implementation allows step-by-step FMU execution and continuous plotting of both datasets, demonstrating the Digital Twin's capability for monitoring and validation.

This study highlights how standardized modeling (FMI/FMU) and open-source tools (Python, FMPy, Matplotlib) can be combined to create a lightweight Digital Twin architecture. Beyond real-time validation, the approach lays the groundwork for future extensions such as fault detection, predictive maintenance, and closed-loop control.

1.1 Background of report

The concept of Digital Twin technology originated from aerospace and manufacturing industries as a method to create a virtual representation of a physical asset that mirrors its real-time behavior. A Digital Twin combines three main elements:

1. The physical system (equipment or process),
2. Its virtual model (a physics-based or data-driven simulation), and
3. The data connection that synchronizes the two.

Through continuous data exchange, Digital Twin can predict performance, detect anomalies, and support optimization or control. Modern implementations rely on sensor networks, Internet of Things (IoT) platforms, and simulation standards such as the Functional Mock-up Interface (FMI), which allows tools like Siemens Amesim to export models as Functional Mock-up Units (FMUs) for use in other environments.

In engineering applications, Digital Twins bridge the gap between theoretical modeling and actual operation. For systems such as pumps, turbines, or propulsion units, they

provide insight into efficiency, transient response, and degradation that would be difficult to obtain experimentally alone. As computing and data acquisition technologies advance, Digital Twins are becoming essential tools for real-time system monitoring, performance validation, and predictive maintenance.

1.2 Objectives of the report

The main objective of this project is to develop and demonstrate a Digital Twin of a laboratory pump system that integrates real-time experimental data with a virtual simulation model.

Specific objectives are to:

- Acquire and calibrate live data from the pump system using NI-DAQ hardware.
- Simulate the system in real time using an Amesim model exported as an FMU and executed in Python via FMPy.
- Visualize and compare measured and simulated parameters—flow rate and pressure difference—in real time.
- Validate the accuracy and responsiveness of the Digital Twin under varying operating conditions.

1.3 Motivation

Pumps are critical components in industrial and marine systems, where performance, reliability, and efficiency are essential. Traditional testing and monitoring methods provide limited insight into dynamic behavior or fault conditions. The Digital Twin concept offers a powerful approach to overcome these limitations by coupling physical measurements with a live simulation model. This enables continuous monitoring, faster diagnostics, and informed decision-making. Developing such a twin for a simple pump system demonstrates how model-based and data-driven approaches can work together in real-time engineering applications.

1.4 Scope

This project focuses on the real-time mirroring of a centrifugal pump system through the integration of sensor data and an FMU-based simulation model.

The scope includes:

- Data acquisition, calibration, and visualization.
- FMU execution and synchronization with live measurements.
- Comparison of flow rate and pressure rise between experimental and simulated results.

The work does not include advanced control design, fault prediction algorithms, or cloud dashboard deployment, although these are identified as future extensions of the Digital Twin framework.

CHAPTER 2: SYSTEM DESCRIPTION

The experimental setup consists of a closed-loop water circulation system designed to study the hydraulic and performance characteristics of a centrifugal pump. The system comprises a single tank, which functions both as the water reservoir (source) and return sink, ensuring a continuous flow cycle.

Water is drawn from the tank through a suction line, passes through the centrifugal pump, and is discharged back into the same tank via a connected piping system. The setup allows for controlled variation of pump speed and flow conditions while monitoring real-time parameters using installed sensors.

Four primary sensors are integrated into the system for data acquisition and Digital Twin synchronization:

1. ***Rotational Speed Sensor (RPM Sensor):***
Measures the angular velocity of the pump shaft, providing data for evaluating pump performance and serving as an input to the FMU model.
2. ***Flow Sensor:***
Installed along the discharge line to measure the volumetric flow rate of the water. This sensor output is used both for real-time monitoring and as a comparative reference for the simulated flow rate in the Digital Twin.
3. ***Suction Pressure Sensor:***
Positioned at the inlet of the pump to record the suction-side pressure. This allows estimation of pressure drop and aids in calculating the total head developed by the pump.
4. ***Discharge Pressure Sensor:***
Mounted at the outlet of the pump to measure discharge pressure. Together with the suction sensor, it enables determination of the differential pressure (ΔP), a key performance indicator for validation of the virtual model.



Figure 1: Physical System

All sensors are connected to a **National Instruments (NI-DAQ)** data acquisition system, which converts the analog voltage signals to digital form for processing in Python. The signals are calibrated into engineering units (rpm, bar, and L/min) using predefined linear relationships.

The entire assembly is supported on a metallic frame, with flexible hoses for suction and discharge lines, and an electrical control panel for power supply and pump operation. This configuration provides a compact, safe, and reconfigurable platform for implementing real-time Digital Twin experiments.

CHAPTER 3: DIGITAL TWIN ARCHITECTURE

The Digital Twin architecture implemented in this project establishes a dynamic connection between the physical pump system, the virtual simulation model, and a cloud-based storage environment. The system integrates three key layers: data acquisition, simulation, and visualization, with data exchange and synchronization managed through Python scripts.

3.1 Overview

The Digital Twin replicates the real-time behavior of the centrifugal pump using live data acquired from sensors measuring flow rate, pump speed (RPM), and suction/discharge pressures. The data is processed, calibrated, and sent to both the simulation model and cloud storage, creating a continuous feedback loop between the physical and digital domains.

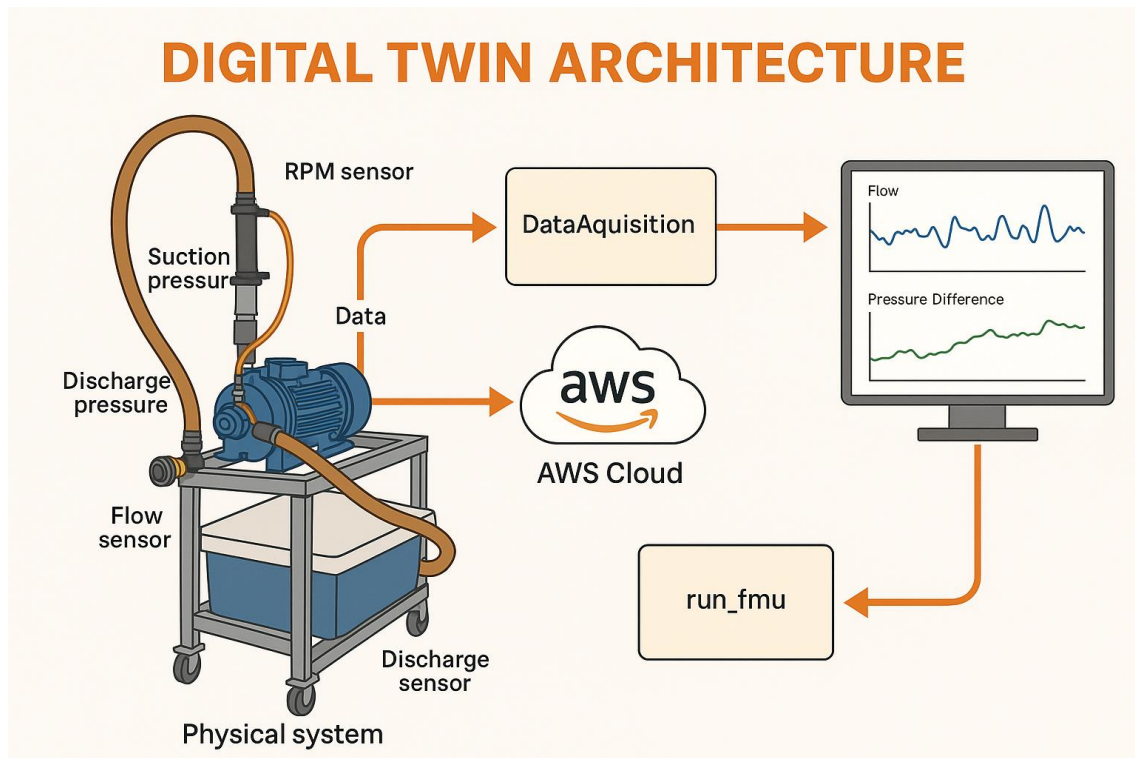


Figure 2 Digital Twin Architecture

3.2 Data Acquisition layer

The python script *DataAquisitionCode.py* handles the real-world data collection process. Sensor signals are read through NI-DAQ hardware, converted into voltage readings, and calibrated into engineering units using linear relationships. The script then stores these values locally and uploads the dataset to an AWS S3 bucket, ensuring secure cloud access and historical data archiving for model validation and analytics.

3.3 Simulation layer

The python script *run_fmu.py* executes the Amesim Functional Mock-up Unit (FMU) model in Python using the FMPy library. The FMU simulates pump behavior based on the acquired input parameters (flow target and pump speed) and outputs virtual responses such as simulated flow rate and pressure increase. These results are plotted and compared with real sensor data to evaluate model accuracy and synchronization.

3.4 Cloud and Visualization Layer

Cloud integration via AWS ensures that measured datasets are preserved for later analysis and remote access. The simulation outputs are visualized using Matplotlib, enabling direct comparison between measured and simulated system responses. This real-time visualization demonstrates the Digital Twin's predictive capability and serves as the foundation for future control and optimization applications.

CHAPTER 4: FMU MODEL DEVELOPMENT AND SIMULATION

4.1 Overview

The Functional Mock-up Unit (FMU) is the core of the Digital Twin's virtual model. It represents the dynamic behavior of the pump system developed in Siemens Amesim and allows co-simulation or model exchange with external tools via the Functional Mock-up Interface (FMI) standard.

In this project, the physical pump was modeled in Amesim and exported as an FMU with two input variables—flowrate target and pump speed (RPM) and two output variables flowrate and pressure increase (ΔP). The FMU was later executed in Python using the FMPy library to simulate the pump's performance and validate it against real measured data.

4.2 System Modeling in Amesim

The pump system was modeled as a hydraulic network in Siemens Amesim, consisting of a reservoir, suction and discharge lines, a centrifugal pump, and a variable restriction (valve).

Sensors were placed in the model at the same physical locations as in the experimental setup—across the suction and discharge sections and along the flow line. The pump component was parameterized according to manufacturer or measured performance data.

The model was configured to receive the pump speed (RPM) and flowrate target as external inputs, enabling it to replicate the varying operational conditions observed during experimentation. The pressure increase (discharge minus suction pressure) and flowrate were defined as the main outputs of interest.

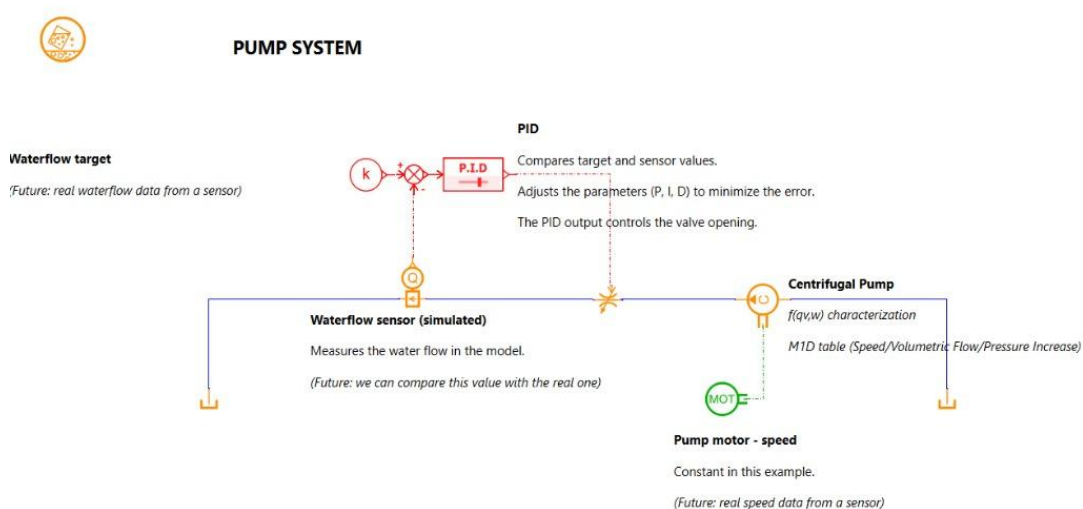


Figure 3 Pump Sytem Amesim model

4.3 Experimental Data for Model Validation

Real-time experimental data were collected from the physical system at increasing pump speeds and different valve openings—from half-open to fully open (three revolutions). For each operating condition, the flowrate and pressure difference were recorded to generate pump performance curves (head vs. flowrate). This dataset provided the baseline for validating the FMU model under realistic operating conditions. The acquired data were saved in CSV format and uploaded to an AWS cloud bucket for secure storage and subsequent simulation use.

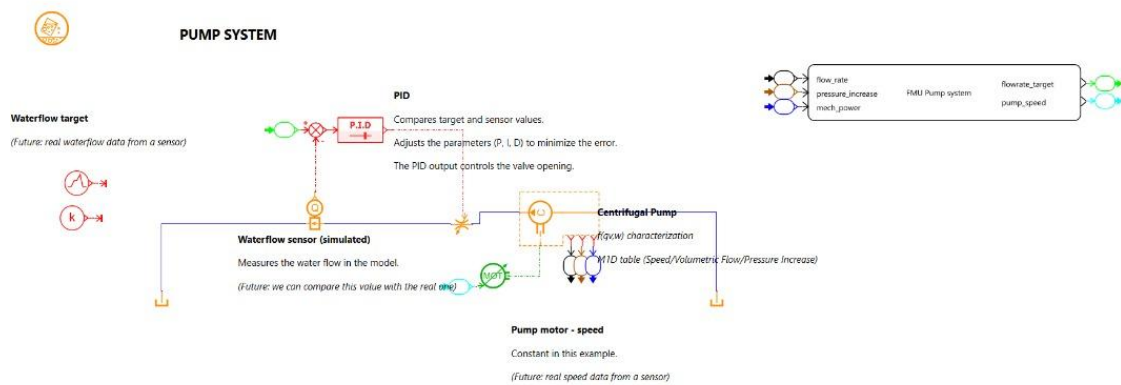


Figure 4 Pump system fmu Amesim model

4.4 FMU Export and Simulation Workflow

After validating the Amesim model, it was exported as an FMU file using the built-in Amesim FMU export utility. The FMU complies with the FMI 2.0 standard, which defines how model variables and interfaces are exposed for external simulation.

The workflow followed these main steps:

1. Model Configuration — Assign input/output connectors for the flowrate target and pump speed; define simulation parameters.
2. FMU Export — From Amesim's Simulation tab, the model was exported as a Co-Simulation FMU.
3. Script Setup — The Python script `run_fmu.py` was configured to:
 - Read and extract the FMU using `read_model_description()` and `extract()`.
 - Define time steps and structured input arrays (time, flowrate target, and pump speed).
 - Execute the FMU simulation using `simulate_fmu()`.

- Plot the output variables (flow_rate and pressure_increase) against time.
4. Simulation Execution — The FMU was simulated using recorded or constant input profiles. The results were plotted for analysis.

4.5 Pump Curve Development

Using the collected pump data as simulation inputs, the FMU generated predicted flowrate and pressure difference outputs. The simulation captured the trend of the physical system—higher speeds and larger valve openings produced higher flowrates and pressure heads, consistent with typical centrifugal pump characteristics.

The simulated pump performance curves were compared with the experimental data to evaluate model fidelity. Minor deviations were observed due to unmodeled losses and sensor calibration limits, but overall, the FMU replicated the real pump behavior effectively.

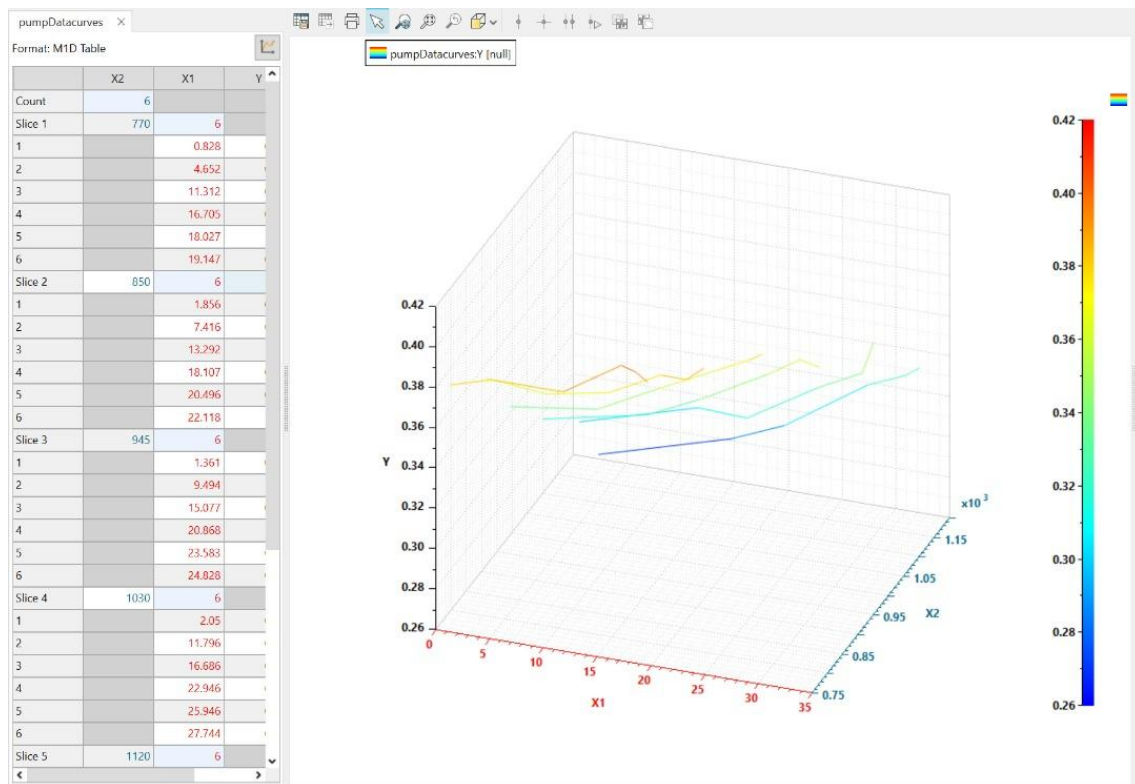


Figure 5 Pump data curves

CHAPTER 5: REAL TIME INTEGRATION

The real-time connection between the physical pump system and its virtual model was implemented using Python through the `Simulation_realtime.py` script. The system continuously acquires live sensor data, runs the Amesim FMU simulation, and visualizes both measured and simulated results, forming the operational Digital Twin.

5.1 Integration Workflow

The workflow follows a continuous loop:

1. Acquire live signals (RPM, suction pressure, discharge pressure, flowrate) via NI-DAQ.
2. Calibrate voltage readings into engineering units using defined equations.
3. Compute the pressure difference ($\Delta P = P_{\text{out}} - P_{\text{in}}$).
4. Feed measured flowrate and RPM into the FMU model.
5. Execute an FMU simulation step and obtain virtual outputs (flowrate and ΔP).
6. Plot real-time graphs comparing measured and simulated data.

5.2 Code Structure

- *Data Acquisition:* Four NI-DAQ analog channels read sensor voltages at 10 Hz. A dummy mode generates synthetic data when no hardware is available.
- *Calibration:* Linear relationships convert voltage signals to RPM, flowrate (L/min), and pressures (bar).
- *FMU Execution:* The FMU receives flowrate and pump speed as inputs and returns simulated flowrate and pressure increase.
- *Visualization:* Matplotlib displays live comparisons in a rolling 60-second window.

5.3 Simulation Modes

The FMU operates in:

- *Co-Simulation (CS):* step-by-step execution using `doStep()`, ideal for real-time synchronization.
- *Model Exchange (ME):* re-simulates from stored input history, used when no internal solver is available.

5.4 Data Buffering and Synchronization

A rolling buffer stores about three minutes of data to maintain smooth visualization. Time steps are enforced at 0.1 s using `time.sleep()`, ensuring consistent synchronization between physical and virtual systems.

CHAPTER 6: RESULTS AND LIMITATIONS

6.1 Results

The Digital Twin successfully established real-time synchronization between the physical pump system and the Amesim FMU simulation, allowing live comparison of measured and simulated parameters. The Python-based interface continuously displayed flowrate and pressure difference (ΔP) trends from both domains.

At various pump speeds and valve openings, the following observations were made:

- **Flowrate (L/min):**
The simulated flow closely followed the measured flow profile across time, maintaining similar dynamic trends. Minor transient discrepancies were observed during sudden flow fluctuations, which correspond to rapid sensor response variations or valve disturbances.
- **Pressure Difference (ΔP in bar):**
The simulated pressure rise captured the general pattern of the measured ΔP , though the amplitude of variation in the simulation was smoother. The physical system showed higher noise due to pressure pulsations and sensor resolution limits.
- **System Responsiveness:**
Both datasets demonstrated consistent behavior with changes in pump speed. Higher RPM resulted in increased flowrate and pressure, aligning with the theoretical performance characteristics of a centrifugal pump.

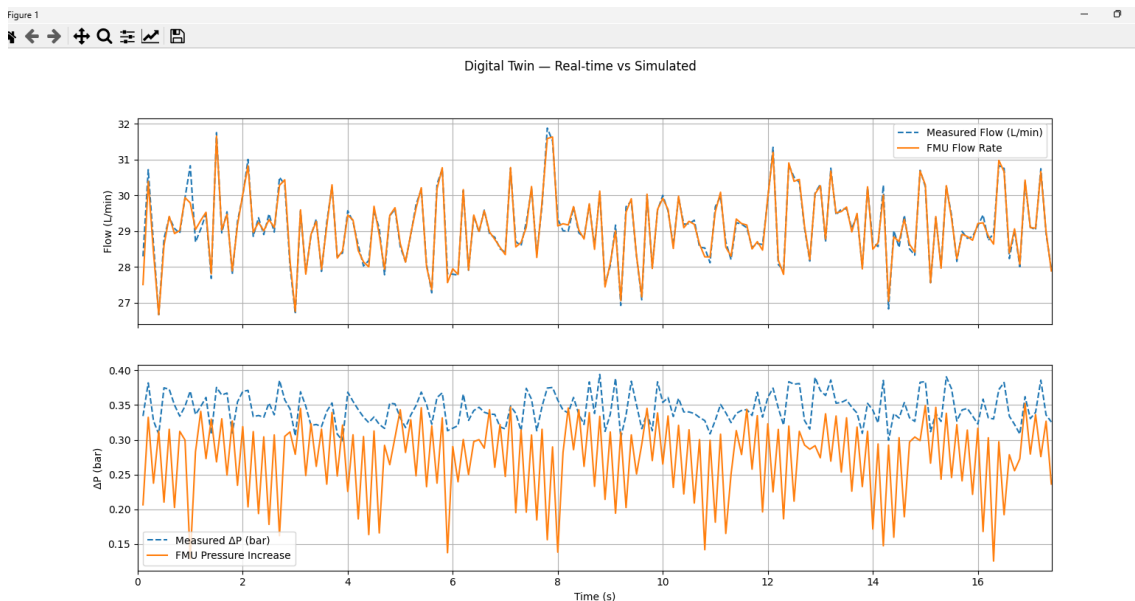


Figure 6 Plot outputs at initial run

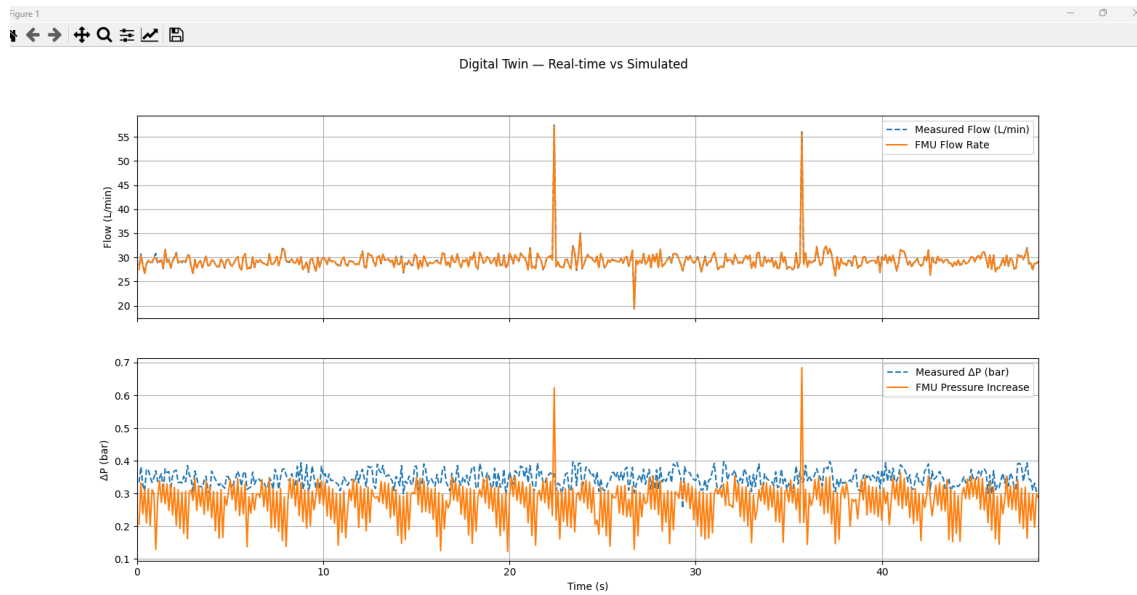


Figure 7 Plot output stabilized

6.2 Limitations of the Model

Despite the strong correlation between measured and simulated data, several limitations were identified:

1. **Simplified Pump Model:**
The Amesim FMU used static performance curves (M1D tables) without incorporating dynamic hydraulic losses, cavitation effects, or nonlinear valve behavior, leading to underestimation of transient variations.
2. **Sensor Noise and Drift:**
The measured data contained small spikes and oscillations, particularly in flow and pressure readings, caused by sensor noise, temperature effects, and analog-to-digital conversion errors.
3. **Latency in Real-Time Execution:**
The Python real-time loop, limited to a 10 Hz sampling rate, introduced slight latency between data acquisition and FMU updates. This minor delay can affect precise transient synchronization.
4. **Model Exchange Constraints:**
For Model Exchange FMUs, re-simulation for each new input set is computationally expensive and unsuitable for fast real-time response. Co-Simulation FMUs perform better but require solver stability tuning.
5. **Calibration Uncertainty:**
The linear calibration equations used for pressure and flow sensors assumed ideal scaling; small deviations in sensor offset can propagate as steady-state bias in measured data.

6. Limited Operating Range:

The system was validated only within moderate speed and valve openings (half-open to fully open). Extreme operating points were not tested to avoid overloading the pump motor or sensors

CHAPTER 7: CONCLUSIONS AND FUTURE WORK

7.1 Conclusions

This project successfully developed a Digital Twin of a centrifugal pump system by integrating real-time sensor data with a virtual model exported from Siemens Amesim as a Functional Mock-up Unit (FMU). The system architecture, implemented in Python, enabled continuous acquisition, simulation, and visualization of measured and simulated parameters, specifically flowrate and pressure difference.

The results demonstrated that the FMU model accurately replicated the physical pump's dynamic behavior under varying speeds and valve openings. The real-time plots confirmed close agreement between measured and simulated data, validating the model's predictive capability within the tested range.

Although minor discrepancies were observed due to sensor noise, calibration uncertainty, and simplified model dynamics, the Digital Twin proved effective as a framework for system monitoring, validation, and future control applications.

In conclusion, this work establishes a functional real-time Digital Twin platform that bridges experimental and virtual domains, laying the foundation for future enhancements in fault detection, predictive maintenance, and closed-loop control of hydraulic systems.

7.2 Future Work

The developed Digital Twin provides a solid foundation for further research and system enhancement. However, several improvements can be implemented to expand its accuracy, functionality, and application scope:

1. **Advanced Dynamic Modeling:** Incorporate nonlinear hydraulic losses, valve dynamics, and transient pump characteristics into the Amesim model to improve simulation fidelity, especially during rapid operational changes.
2. **Automated Parameter Calibration:** Implement adaptive algorithms to automatically tune model parameters based on live data, reducing discrepancies between measured and simulated outputs.
3. **Predictive Maintenance Analytics:** Integrate machine learning or statistical health models to predict anomalies, detect degradation, and schedule maintenance using historical and live sensor data.